

To-Do List Manager en C++

Documentación técnica del código

Fase 1 — CRUD en memoria con Programación Orientada a Objetos

-Gerardo Avalos Sánchez

- Rubén Jesús García Torres

- Daniel Ernesto Garrido Sánchez

- Joshua Isacar Romero Sánchez

Universidad Autónoma de Guadalajara

C++17 · g++ · 10 archivos fuente · 39 pruebas unitarias

Índice

Si el índice aparece vacío, haz clic derecho sobre él y elige "Actualizar campos" (F9).

1. Panorama general

Este documento explica, archivo por archivo y método por método, el programa To-Do List Manager escrito en C++17. El programa es una aplicación de consola que permite crear, consultar, actualizar y eliminar tareas guardadas en memoria, cada una con título, descripción, estado de progreso y fecha de creación.

El código está repartido en cuatro capas con responsabilidades separadas. Esa separación es la decisión de diseño más importante del proyecto y es la que hace posible probar la lógica sin simular el teclado.

Capa	Archivos	Responsabilidad
Modelo	Task, TaskList	Guarda los datos y las reglas sobre ellos. No imprime ni lee nada.
Entrada	InputValidator	Lee de la consola y garantiza que el dato devuelto sea válido.
Interfaz	Menu	Dibuja pantallas y hace preguntas. Es el único que usa <code>std::cout</code> .
Arranque	main	Conecta las piezas: arma el menú y corre el bucle principal.
Pruebas	tests/	Verifica el modelo de forma automática, sin intervención humana.

La regla que ordena todo: las dependencias apuntan hacia adentro. Menu conoce a TaskList, TaskList conoce a Task, y Task no conoce a nadie. Ninguna clase del modelo incluye `<iostream>`, así que ninguna puede imprimir por accidente.

1.1 Estructura de carpetas

```
To-Do-List/
├── .vscode/
│   ├── tasks.json          Tareas de compilación y pruebas
│   ├── launch.json         Configuraciones de depuración
│   └── c_cpp_properties.json
├── src/
│   ├── Task.hpp / Task.cpp
│   ├── TaskList.hpp / TaskList.cpp
│   ├── InputValidator.hpp / InputValidator.cpp
│   ├── Menu.hpp / Menu.cpp
│   └── main.cpp
```

```
├── tests/
│   ├── TestFramework.hpp / TestFramework.cpp
│   ├── Tests.hpp
│   ├── TaskTests.cpp
│   ├── TaskListTests.cpp
│   └── TestMain.cpp
└── build/                               Ejecutables (ignorado por Git)
```

1.2 Convenciones del código

- Tipos en PascalCase (Task, TaskList, MenuOption); métodos y variables en camelCase (getTaskCount, creationDate).
- Métodos que consultan sin modificar se marcan const; los parámetros de tipo std::string se reciben como const std::string& para evitar copias.
- Cada método lleva un comentario Doxygen con @brief, @param y @return en el archivo .cpp, donde vive el cuerpo.
- Todos los encabezados usan guardas de inclusión #ifndef / #define / #endif.
- Los nombres, comentarios y mensajes en pantalla están en inglés; la documentación, en español.

2. La clase Task

Task representa una sola tarea. Es la unidad de información del programa y no depende de ninguna otra clase del proyecto.

2.1 El enumerado TaskStatus

```
enum class TaskStatus {  
    Pending = 1,  
    InProgress = 2,  
    Completed = 3  
};  
  
constexpr int taskStatusCount = 3;
```

Se usa `enum class` y no un `enum` clásico por dos razones. La primera es que sus valores viven dentro del espacio de nombres del tipo: hay que escribir `TaskStatus::Pending`, lo que evita choques de nombres. La segunda es que no se convierte a entero de forma automática, así que el compilador impide errores como asignarle un 7 a una variable de estado.

Los valores se numeran desde 1 a propósito: coinciden con las opciones que ve el usuario en pantalla, así que `static_cast<int>(TaskStatus::Pending)` imprime el número correcto sin tablas de conversión. La constante `taskStatusCount` evita que el número 3 aparezca escrito a mano en otros archivos.

2.2 Atributos

Atributo	Tipo	Qué guarda
title	std::string	Nombre corto de la tarea. Lo captura el usuario y no puede quedar vacío.
description	std::string	Texto detallado de la tarea. También obligatorio al crear.
status	TaskStatus	Estado de progreso: pendiente, en progreso o completada.
creationDate	std::string	Momento de creación en formato "AAAA-MM-DD HH:MM". Lo genera la clase, no el usuario.

Los cuatro son privados. El acceso desde afuera pasa obligatoriamente por los métodos públicos, que es lo que significa el encapsulamiento en la práctica: la clase decide qué se puede leer y qué se puede cambiar.

2.3 Constructor

```
Task::Task(const std::string& title,
           const std::string& description,
           TaskStatus status)
: title(title),
  description(description),
  status(status),
  creationDate(buildCurrentTimestamp()) {
}
```

Se usa lista de inicialización (los dos puntos después de la firma) en lugar de asignaciones dentro de las llaves. La diferencia real: con asignaciones, cada `std::string` se construye vacío y después se le copia el valor, es decir dos operaciones; con lista de inicialización se construye directamente con su valor final, una sola operación.

El constructor solo recibe tres parámetros, no cuatro. La fecha no se pide: la genera la propia clase. Esto hace imposible crear una tarea con una fecha de creación falsa, y es la razón por la que `creationDate` no tiene setter.

Función auxiliar `buildCurrentTimestamp`

```
namespace {

constexpr std::size_t dateBufferSize = 20;

std::string buildCurrentTimestamp() {
    const std::time_t currentTime = std::time(nullptr);
    const std::tm* localTime = std::localtime(&currentTime);

    char buffer[dateBufferSize] = { 0 };
    std::strftime(buffer, dateBufferSize, "%Y-%m-%d %H:%M", localTime);

    return std::string(buffer);
}

} // namespace
```

Está dentro de un namespace anónimo, lo que la vuelve privada del archivo `Task.cpp`: ningún otro archivo puede llamarla ni chocar con un nombre igual. `std::strftime` formatea la hora local en "AAAA-MM-DD HH:MM", un formato que además se ordena alfabéticamente igual que cronológicamente, útil para la Fase 3 con base de datos.

2.4 Métodos consultores (getters)

Método	Devuelve	Descripción
getTitle() const	const std::string&	Entrega el título sin copiarlo.
getDescription() const	const std::string&	Entrega la descripción sin copiarla.
getStatus() const	TaskStatus	Entrega el estado. Al ser un valor pequeño se devuelve por copia.
getCreationDate() const	const std::string&	Entrega la fecha de creación.

Los tres que devuelven texto lo hacen como `const std::string&`: una referencia, no una copia. El `const` del principio impide que quien la reciba modifique el atributo, y el `const` del final promete que el método no cambia el objeto. Ese segundo `const` es el que permite llamarlos sobre una tarea constante, como ocurre dentro de `Menu::displayTasks`.

2.5 Métodos modificadores (setters)

Método	Descripción
setTitle(const std::string&)	Reemplaza el título por el texto recibido.
setDescription(const std::string&)	Reemplaza la descripción.
setStatus(TaskStatus)	Reemplaza el estado de progreso.

Hay tres setters, no cuatro: `creationDate` no tiene ninguno. Esa ausencia es intencional y es la respuesta a la pregunta "¿por qué no documentaste un setter para la fecha?". La fecha de creación es un hecho histórico; permitir cambiarla sería permitir falsificar el registro. La prueba `testCreationDateIsReadOnly` verifica justamente eso.

Dentro de los setters se escribe `this->title = title` porque el parámetro se llama igual que el atributo. El apuntador `this` desambigua: a la izquierda el atributo del objeto, a la derecha el parámetro.

2.6 Funciones libres de conversión

```
std::string statusToString(TaskStatus status);  
TaskStatus statusFromInt(int value);
```

No son métodos de la clase porque no necesitan acceso a los atributos privados de ninguna tarea: convierten un `TaskStatus` suelto. Declararlas fuera de la clase mantiene la interfaz de `Task` concentrada en lo que una tarea sabe hacer.

`statusToString` traduce el estado al texto que se imprime ("Pending", "In progress", "Completed"). `statusFromInt` hace el camino inverso, del número tecleado al estado, y cuando el número no corresponde a ninguno devuelve `TaskStatus::Pending` en lugar de fallar. Ese valor de respaldo nunca se alcanza en la práctica porque `InputValidator` ya filtró el rango, pero deja la función segura si algún día se la llama desde otro lado.

3. La clase TaskList

TaskList guarda la colección de tareas y ofrece las cuatro operaciones del CRUD. Internamente usa un `std::vector<Task>`, que crece solo conforme se agregan tareas y libera su memoria automáticamente al terminar el programa.

3.1 Método privado isValidIndex

```
bool TaskList::isValidIndex(int index) const {  
    return index >= 0 && index < this->getTaskCount();  
}
```

Las cuatro operaciones que reciben una posición necesitan la misma comprobación. Antes estaba copiada y pegada cuatro veces; extraerla a un método privado significa que si la regla cambia, se corrige en un solo lugar. Es privado porque es un detalle interno: nadie fuera de la clase necesita preguntarle si un índice es válido.

3.2 addTask

```
void TaskList::addTask(const std::string& title,  
                      const std::string& description,  
                      TaskStatus status) {  
    this->tasks.push_back(Task(title, description, status));  
}
```

Recibe los datos sueltos y construye la tarea aquí dentro; quien llama no necesita conocer la clase Task. La tarea nueva siempre se agrega al final, así que el orden de la lista es el orden de creación y los números que ve el usuario se mantienen estables mientras no borre nada.

3.3 Los tres métodos de actualización

Método	Parámetros	Devuelve
updateTaskTitle	int index, const std::string& title	true si la tarea existe
updateTaskDescription	int index, const std::string& description	true si la tarea existe
updateTaskStatus	int index, TaskStatus status	true si la tarea existe

```
bool TaskList::updateTaskTitle(int index, const std::string& title) {  
    if (!this->isValidIndex(index)) {  
        return false;  
    }  
}
```

```

        this->tasks[index].setTitle(title);
        return true;
    }

```

Los tres siguen la misma estructura: validar y salir temprano si el índice no sirve, o modificar y confirmar. Ese patrón de "salida temprana" evita anidar if dentro de if y deja el camino normal sin sangría extra.

Devuelven bool y no void para que quien llama pueda distinguir entre "se actualizó" y "no existía esa tarea", y mostrar el mensaje correspondiente. Son tres métodos separados en vez de uno que reciba todo: así la interfaz puede cambiar solo el estado sin tocar el texto, que es exactamente lo que hace la opción de actualizar cuando el usuario presiona Enter para conservar el título.

3.4 deleteTask

```

bool TaskList::deleteTask(int index) {
    if (!this->isValidIndex(index)) {
        return false;
    }

    this->tasks.erase(this->tasks.begin() + index);
    return true;
}

```

erase necesita un iterador, no un número, por eso se escribe begin() + index: se parte del inicio del vector y se avanza tantas posiciones. Al borrar, las tareas siguientes se recorren una posición hacia adelante y se reenumeran en la pantalla; la prueba testDeleteTaskShiftsTheRest verifica ese corrimiento.

3.5 Métodos de consulta

Método	Devuelve	Descripción
isEmpty() const	bool	Indica si no hay tareas guardadas. Se lee mejor que comparar el conteo contra cero.
getTaskCount() const	int	Cantidad de tareas. Convierte el size_t del vector a int con static_cast.
getTasks() const	const std::vector<Task>&	Acceso de solo lectura al vector interno.

getTasks es el método más delicado de la clase. Devuelve una referencia constante: quien la recibe puede recorrer las tareas e imprimirlas, pero no puede agregar, borrar ni modificar ninguna, porque todo lo que obtiene a través de ella es constante. Si devolviera el vector

por copia, imprimir la lista duplicaría todas las tareas en memoria; si lo devolviera sin `const`, la interfaz podría modificar los datos a espaldas de la clase y el encapsulamiento se perdería.

`getTaskCount` usa `static_cast<int>` porque `size()` devuelve `std::size_t`, un entero sin signo. Sin la conversión explícita, el compilador emite una advertencia con `-Wall`, y comparar con signo contra sin signo es una fuente clásica de errores cuando el valor es cero.

4. La clase InputValidator

InputValidator concentra toda la lectura de teclado. Su promesa es simple: si un método devuelve, el dato es válido. Nadie más en el programa tiene que revisar lo que tecleó el usuario.

Todos sus métodos son `static`, es decir se llaman con `InputValidator::readLine(...)` sin crear un objeto. Tiene sentido porque la clase no guarda ningún estado entre llamadas: es un conjunto de herramientas, no un objeto con datos propios.

4.1 clearInputBuffer (privado)

```
void InputValidator::clearInputBuffer() {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

`clear()` reinicia las banderas de error del flujo, que quedan activas cuando el usuario escribe letras donde se esperaba un número; mientras estén encendidas, toda lectura posterior falla en cadena. `ignore(max, '\n')` descarta lo que quede de la línea, incluido el salto de línea final.

Este método es la solución a un error que costó mucho tiempo detectar. El operador `>>` deja el salto de línea en el buffer; si después se llama a `getline`, este encuentra ese salto y devuelve una cadena vacía sin esperar a que el usuario escriba nada. Por eso `clearInputBuffer` se ejecuta en todas las vueltas del ciclo, tanto cuando la lectura falla como cuando tiene éxito.

4.2 readIntInRange

```
int InputValidator::readIntInRange(int minValue, int maxValue) {
    int value = 0;
    bool isValid = false;

    do {
        std::cin >> value;

        if (std::cin.fail()) {
            std::cout << ">> Invalid input. Please enter a number." << std::endl;
        } else if (value < minValue || value > maxValue) {
            std::cout << ">> Value out of range. Please try again." << std::endl;
        } else {
            isValid = true;
        }

        clearInputBuffer();
    } while (!isValid);

    return value;
}
```

```

    } while (!isValid);

    return value;
}

```

Parámetro	Significado
minValue	Valor mínimo aceptado, inclusive.
maxValue	Valor máximo aceptado, inclusive.

Usa do-while porque la lectura tiene que ocurrir al menos una vez antes de poder juzgarla. Distingue dos fallas con mensajes distintos: texto donde iba un número, y número fuera del rango. Solo sale del ciclo cuando el valor sirve, de modo que el resto del programa nunca recibe una opción inválida.

Este método es el que hace innecesario validar en otros lados. Cuando el menú pregunta por el número de tarea con el rango de 1 al total, es imposible que llegue un índice inexistente a TaskList.

4.3 readLine y readNonEmptyLine

```

std::string InputValidator::readLine(const std::string& prompt) {
    std::string line;

    std::cout << prompt;
    std::getline(std::cin, line);

    return line;
}

std::string InputValidator::readNonEmptyLine(const std::string& prompt) {
    std::string line;

    do {
        line = readLine(prompt);

        if (line.empty()) {
            std::cout << ">> Text cannot be empty. Please try again." << std::endl;
        }
    } while (line.empty());

    return line;
}

```

Se usa `std::getline` y no `std::cin >> texto` porque el operador se detiene en el primer espacio: con él, "Comprar leche" se guardaría como "Comprar". `getline` lee la línea completa.

Los dos métodos existen porque el programa necesita dos comportamientos distintos. Al crear una tarea, el título y la descripción son obligatorios y se usa `readNonEmptyLine`. Al actualizar, una línea vacía es una respuesta legítima que significa "déjalo como está", y ahí se usa `readLine`. `readNonEmptyLine` se apoya en `readLine` en vez de repetir el código de lectura.

5. La clase Menu

Menu es la interfaz de usuario completa: todo lo que se imprime y todo lo que se pregunta pasa por aquí. Ninguna otra clase incluye `<iostream>` salvo `InputValidator`, que es su ayudante de lectura.

5.1 La estructura MenuOption

```
struct MenuOption {  
    std::string label;  
    std::function<void()> action;  
};
```

Esta estructura de dos campos es el corazón del diseño del menú. Cada opción guarda el texto que se muestra y la acción que ejecuta, unidos en un mismo objeto. Es una struct y no una class porque no tiene lógica ni datos que esconder: solo agrupa dos valores públicos.

`std::function<void()>` es una variable que guarda una función, igual que `std::string` guarda texto. El `void()` describe la forma de la función que acepta: no recibe parámetros y no devuelve nada. Para ejecutarla se escribe `action()`.

5.2 addOption

```
void Menu::addOption(const std::string& label, std::function<void()> action) {  
    this->options.push_back({ label, action });  
}
```

Registra una opción nueva. La posición en la que queda determina el número con el que se muestra: la primera registrada es la [1]. Agregar una funcionalidad al programa es una sola llamada a este método, sin tocar ningún otro archivo.

5.3 render (privado)

```
for (std::size_t i = 0; i < this->options.size(); ++i) {  
    const bool isLastOption = (i + 1 == this->options.size());  
  
    if (isLastOption) {  
        std::cout << std::endl;  
    }  
  
    std::cout << "  [" << (i + 1) << "] " << this->options[i].label << std::endl;  
}
```

Dibuja el encabezado y recorre las opciones registradas imprimiendo su etiqueta. Esto es lo que impide que el menú se desincronice: el texto en pantalla sale de la misma lista que

ejecuta las acciones, así que no puede haber una opción visible sin acción ni una acción sin opción visible.

La variable `isLastOption` existe solo para darle nombre a la condición: separa visualmente la salida (Exit) de las operaciones del CRUD. Es más legible que meter la comparación directamente dentro del `if`.

5.4 displayMenu

```
int Menu::displayMenu() const {
    system("clear");
    this->render();

    return InputValidator::readIntInRange(1, static_cast<int>(this-
>options.size()));
}
```

Limpia la pantalla, dibuja el menú y lee una opción válida. El rango que le pasa al validador es 1 al número de opciones registradas, calculado en el momento: si mañana se agrega una sexta opción, el rango se ajusta solo.

`system("clear")` funciona en Linux y macOS; en Windows el comando equivalente es `"cls"`. Es la única línea del proyecto que depende del sistema operativo.

5.5 execute

```
void Menu::execute(int choice) const {
    this->options.at(choice - 1).action();
}
```

Este método es el que sustituye al `switch`. Toma el número que eligió el usuario, resta uno para pasar de la numeración visible (desde 1) al índice del vector (desde 0), y ejecuta la acción guardada en esa posición.

Se usa `at()` y no los corchetes porque `at()` lanza una excepción si el índice se sale del vector, mientras que los corchetes provocarían comportamiento indefinido. Dado que `displayMenu` ya validó el rango, es una red de seguridad más que una necesidad.

5.6 displayTasks

```
void Menu::displayTasks(const TaskList& taskList) const {
    if (taskList.isEmpty()) {
        std::cout << ">> No tasks available." << std::endl;
        return;
    }

    const std::vector<Task>& tasks = taskList.getTasks();
```

```

std::cout << "\n--- TASK LIST (" << tasks.size() << ") ---" << std::endl;

for (std::size_t i = 0; i < tasks.size(); ++i) {
    const Task& task = tasks[i];

    std::cout << "  [" << (i + 1) << "] " << task.getTitle() << std::endl;
    std::cout << "      Description: " << task.getDescription() << std::endl;
    std::cout << "      Status: " << statusToString(task.getStatus())
        << " | Created: " << task.getCreationDate() << std::endl;
}
}

```

Recibe la lista como `const TaskList&`: por referencia para no copiar todas las tareas, y constante para dejar claro que solo las va a leer. Dentro, `const Task& task = tasks[i]` le pone nombre a la tarea de la vuelta actual y evita repetir `tasks[i]` cuatro veces.

La numeración que ve el usuario es $i + 1$, mientras que el vector trabaja desde 0. Esa diferencia de uno es la que después convierte `askTaskIndex`, y es el error más común en este tipo de programas.

5.7 Métodos de pregunta

Método	Devuelve	Descripción
<code>askTitle()</code>	<code>std::string</code>	Pide el título y rechaza texto vacío. Se usa al crear.
<code>askTitleOrKeep()</code>	<code>std::string</code>	Pide un título nuevo; cadena vacía significa conservar el actual.
<code>askDescription()</code>	<code>std::string</code>	Pide la descripción y rechaza texto vacío.
<code>askDescriptionOrKeep()</code>	<code>std::string</code>	Pide una descripción nueva; vacío conserva la actual.
<code>askStatus()</code>	<code>TaskStatus</code>	Muestra los tres estados y lee el elegido.
<code>askTaskIndex(const TaskList&)</code>	<code>int</code>	Pide el número de tarea y devuelve el índice base cero.

```

TaskStatus Menu::askStatus() const {
    std::cout << std::endl;
    std::cout << "  [" << static_cast<int>(TaskStatus::Pending) << "] Pending"
<< std::endl;
    std::cout << "  [" << static_cast<int>(TaskStatus::InProgress) << "] In
progress" << std::endl;
}

```

```

        std::cout << " [" << static_cast<int>(TaskStatus::Completed) << "] Completed"
        << std::endl;
        std::cout << "Select a progress state: ";

        return statusFromInt(InputValidator::readIntInRange(1, taskStatusCount));
    }

```

Los números que se imprimen se derivan del propio enumerado en lugar de escribirse a mano. Si algún día se agrega un cuarto estado, basta con declararlo, actualizar `taskStatusCount` y agregar una línea: los números no se pueden desalinear con los valores reales.

```

int Menu::askTaskIndex(const TaskList& taskList) const {
    std::cout << "\nTask number: ";

    return InputValidator::readIntInRange(1, taskList.getTaskCount()) - 1;
}

```

Este método concentra dos responsabilidades que antes estaban repartidas. Valida el rango contra el número real de tareas, y hace la resta de uno que traduce de la numeración del usuario a la del vector. Gracias a él, esa resta aparece una sola vez en todo el proyecto en lugar de repetirse en cada acción, que es donde se colaban los errores por uno.

5.8 showMessage y pause

```

void Menu::pause() const {
    std::cout << "\n>> Press Enter to continue...";
    std::cin.get();
}

```

`pause` resuelve un problema de uso: como `displayMenu` limpia la pantalla al inicio de cada vuelta, sin esta pausa el resultado de la operación se borraría antes de que el usuario alcanzara a leerlo. `cin.get()` espera un solo carácter, que es el Enter.

6. El archivo main.cpp

main.cpp no contiene lógica de negocio ni impresiones directas. Su único trabajo es armar el programa: crear los objetos, registrar las acciones y correr el bucle.

6.1 Por qué no hay switch

El diseño original usaba un switch sobre la opción elegida. Con ese esquema, agregar una opción obligaba a tocar tres lugares distintos: el texto del menú, el rango del validador y el case correspondiente. Olvidar cualquiera de los tres produce un menú inconsistente, y el compilador no avisa.

En el diseño actual cada opción se registra junto con su acción, así que el texto y el comportamiento viajan juntos y no pueden separarse. Agregar una funcionalidad es una sola llamada a `addOption`. Es una aplicación del principio abierto/cerrado: el programa queda abierto a extensión y cerrado a modificación.

6.2 Las funciones de registro

```
void registerCreateOption(Menu& menu, TaskList& taskList) {
    menu.addOption("Create a task", [&menu, &taskList]() {
        menu.showMessage("-> Create a task");

        const std::string title = menu.askTitle();
        const std::string description = menu.askDescription();
        const TaskStatus status = menu.askStatus();

        taskList.addTask(title, description, status);
        menu.showMessage("\n>> Task created successfully.");
    });
}
```

Función	Qué registra
<code>registerCreateOption</code>	Pide título, descripción y estado, y guarda la tarea nueva.
<code>registerReadOption</code>	Muestra la lista completa de tareas.
<code>registerUpdateOption</code>	Permite cambiar título, descripción y estado de una tarea existente.
<code>registerDeleteOption</code>	Elimina la tarea que elija el usuario.
<code>registerExitOption</code>	Apaga la bandera running para terminar el bucle.

Hay una función por acción en lugar de un main gigante. Cada una hace una sola cosa, cabe en una pantalla y se entiende por su nombre. El orden de las cinco llamadas dentro de main define el orden del menú.

6.3 Las lambdas y la captura

El bloque `[&menu, &taskList]() { ... }` es una función anónima, o lambda. Los corchetes del inicio son la lista de captura: dicen qué variables de afuera puede usar la función. El símbolo `&` significa que las toma por referencia, es decir trabaja sobre los objetos originales y no sobre copias.

Esa distinción es crítica. Si se capturaran por valor, cada acción tendría su propia copia de la lista y las tareas creadas desaparecerían al terminar la acción. Se capturan por nombre y no con el atajo `[&]` porque así queda explícito de qué depende cada acción.

En la acción de salida se captura la bandera `running` por referencia, y por eso al apagarla desde dentro de la lambda el bucle de main se entera y termina.

6.4 La acción de actualizar

```
const int index = menu.askTaskIndex(taskList);
const std::string title = menu.askTitleOrKeep();
const std::string description = menu.askDescriptionOrKeep();
const TaskStatus status = menu.askStatus();

if (!title.empty()) {
    taskList.updateTaskTitle(index, title);
}

if (!description.empty()) {
    taskList.updateTaskDescription(index, description);
}

if (taskList.updateTaskStatus(index, status)) {
    menu.showMessage("\n>> Task updated successfully.");
} else {
    menu.showMessage("\n>> Could not update the task.");
}
```

La convención de "cadena vacía significa conservar" se interpreta aquí, en la capa de interfaz, y no dentro de `TaskList`. La clase de datos no sabe nada de esa convención: ella solo recibe un texto y lo guarda. Mantener las convenciones de la interfaz fuera del modelo es lo que permite que el modelo se pueda reutilizar en la Fase 3 con una base de datos sin arrastrar decisiones de la consola.

6.5 El bucle principal

```
int main() {
    TaskList taskList;
    Menu menu;
    bool running = true;

    registerCreateOption(menu, taskList);
    registerReadOption(menu, taskList);
    registerUpdateOption(menu, taskList);
    registerDeleteOption(menu, taskList);
    registerExitOption(menu, running);

    while (running) {
        const int choice = menu.displayMenu();
        menu.execute(choice);

        if (running) {
            menu.pause();
        }
    }

    return 0;
}
```

El bucle tiene tres pasos: mostrar el menú y leer la opción, ejecutar la acción, y pausar. La pausa se condiciona a `running` para que al salir el programa no pida un `Enter` extra después del mensaje de despedida.

`taskList` y `menu` se crean en el `stack`. No hay `new` ni `delete` en todo el proyecto: el vector administra su propia memoria y los objetos se destruyen solos al terminar `main`. Esto elimina de raíz las fugas de memoria.

7. La carpeta tests

Las pruebas verifican de forma automática que el modelo se comporte como se espera. Se compilan en un ejecutable aparte, `build/tests`, y no dependen de ninguna biblioteca externa.

7.1 Qué se prueba y qué no

Solo se compilan `Task.cpp` y `TaskList.cpp` junto con los archivos de tests. `Menu.cpp` e `InputValidator.cpp` quedan fuera porque leen de `std::cin` y no se pueden ejecutar sin simular el teclado; `main.cpp` queda fuera porque tendría un segundo `main` y el enlazador fallaría.

Que el modelo se pueda probar sin la interfaz no es casualidad: es la consecuencia directa de haber mantenido `Task` y `TaskList` libres de impresiones y lecturas. Esa comprobación es, en sí misma, la prueba de que la separación de capas funcionó.

7.2 TestFramework

Método	Uso
<code>startSection(nombre)</code>	Imprime el título de un grupo de verificaciones.
<code>checkTrue(condición, nombre)</code>	Verifica que una condición se cumpla.
<code>checkFalse(condición, nombre)</code>	Verifica que una condición no se cumpla.
<code>checkEqualInt(obtenido, esperado, nombre)</code>	Compara enteros y reporta ambos si difieren.
<code>checkEqualText(obtenido, esperado, nombre)</code>	Compara cadenas y reporta ambas si difieren.
<code>printSummary()</code>	Imprime los totales y devuelve 0 si todo pasó, 1 si algo falló.

Los contadores `passedCount` y `failedCount` son atributos estáticos privados: existen una sola vez para toda la clase, no uno por objeto, y se inicializan en el `.cpp` con la línea `int TestFramework::passedCount = 0`.

El valor que devuelve `printSummary` se convierte en el código de salida del programa. Un cero significa éxito y cualquier otro número significa error, que es la convención que usan las herramientas de integración continua para detener una entrega cuando las pruebas fallan.

7.3 Estructura de un archivo de pruebas

```
void testDeleteTaskShiftsTheRest() {
    TaskList taskList = buildSampleList();

    TestFramework::checkTrue(taskList.deleteTask(0), "deleteTask accepts a valid
index");
    TestFramework::checkEqualInt(taskList.getTaskCount(), 1, "deleteTask decreases
the count");
    TestFramework::checkEqualText(taskList.getTasks()[0].getTitle(), "Second",
"The surviving task moves to the front");

    TestFramework::checkTrue(taskList.deleteTask(0), "The last task can be
deleted");
    TestFramework::checkTrue(taskList.isEmpty(), "The list ends up empty");
    TestFramework::checkFalse(taskList.deleteTask(0), "An empty list rejects
deletions");
}
```

Cada prueba sigue el mismo esquema: preparar una situación conocida, ejecutar la operación y comprobar el resultado. La función auxiliar `buildSampleList` arma una lista con dos tareas conocidas y está dentro de un namespace anónimo, de modo que cada prueba parte de un estado limpio y no se ve afectada por lo que hicieron las anteriores.

7.4 Cobertura actual

Grupo	Verificaciones	Qué cubre
Task	15	Constructor, getters, setters, fecha de solo lectura y conversiones de estado.
TaskList	24	Lista vacía, alta, las tres actualizaciones, rechazo de índices inválidos y baja con corrimiento.
Total	39	Todas pasan.

Las pruebas no solo comprueban los casos correctos. Varias verifican que los métodos rechacen índices negativos o inexistentes y que, al rechazarlos, dejen la lista intacta. Probar los casos límite es lo que distingue una prueba útil de una que solo confirma lo evidente.

8. Compilación y entorno

8.1 Comandos

```
mkdir -p build

# Aplicación
g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o build/main
./build/main

# Pruebas
g++ -std=c++17 -Wall -Wextra -pedantic src/Task.cpp src/TaskList.cpp tests/*.cpp -o
build/tests
./build/tests
```

Bandera	Para qué sirve
-std=c++17	Fija el estándar del lenguaje. Sin ella, el compilador podría usar uno más viejo.
-Wall	Activa las advertencias comunes: variables sin usar, comparaciones entre signo y sin signo.
-Wextra	Agrega advertencias adicionales, como parámetros que nunca se utilizan.
-pedantic	Exige apego estricto al estándar y marca extensiones propias del compilador.
-g	Incluye información de depuración para poder usar gdb y poner puntos de ruptura.

El proyecto compila sin una sola advertencia con las cuatro primeras banderas activas. Ese es el estándar que conviene sostener: una advertencia ignorada hoy es el error difícil de encontrar de la próxima semana.

8.2 Configuración de VS Code

Archivo	Contenido
tasks.json	Tareas para compilar la aplicación, compilar y ejecutar, y correr las pruebas. Todas crean la carpeta build si no existe.
launch.json	Dos configuraciones de depuración: una para build/main y otra para build/tests.
c_cpp_properties.json	Ruta de encabezados y estándar del lenguaje para IntelliSense.

Las tareas incluyen `mkdir -p build` porque la carpeta está ignorada por Git: al clonar el repositorio no existe, y sin ese comando la primera compilación falla en cualquier máquina nueva del equipo.

En `launch.json` la clave `"console": "integratedTerminal"` es indispensable. Con la consola de depuración predeterminada, `std::cin` no recibe lo que se teclea y el programa parece colgado al llegar al menú.

9. Decisiones de diseño

Esta sección reúne las decisiones que conviene poder justificar en la revisión, con el motivo detrás de cada una.

9.1 Separación entre modelo e interfaz

Task y TaskList no imprimen ni leen nada; Menu no guarda tareas. La consecuencia práctica es que el modelo se puede probar automáticamente y que en la Fase 3 se podrá cambiar el almacenamiento en memoria por una base de datos sin tocar una sola línea de la interfaz.

9.2 Opciones de menú con acción incorporada

Sustituir el switch por una lista de pares texto-acción elimina la posibilidad de que el menú mostrado y el comportamiento real se desincronicen, y reduce a una línea el costo de agregar una opción.

9.3 Validación en un solo lugar

Toda la lectura de teclado pasa por InputValidator, que nunca devuelve un dato inválido. Por eso el resto del código no está lleno de comprobaciones defensivas y se lee de corrido.

9.4 Índices desde cero en el modelo, desde uno en pantalla

El vector trabaja con índices desde cero y el usuario ve números desde uno. La traducción ocurre en un único método, askTaskIndex, en lugar de repetirse en cada acción.

9.5 Sin memoria dinámica

No hay new ni delete: los objetos viven en el stack y el vector administra su propio almacenamiento. Es la recomendación moderna de C++ y elimina por completo las fugas de memoria.

9.6 Fecha de creación sin setter

Los cuatro atributos de Task tienen consultor, pero solo tres tienen modificador. La fecha de creación es un dato histórico y la clase no ofrece manera de alterarla.

9.7 Camino hacia las fases siguientes

Fase	Qué se agrega	Qué ya está listo
Fase 2	Herencia y polimorfismo: tipos de	Task ya encapsula sus datos; bastará declarar

Fase	Qué se agrega	Qué ya está listo
	tarea especializados.	sus métodos virtuales y derivar.
Fase 3	Persistencia en base de datos.	TaskList concentra el almacenamiento; sustituir el vector no afecta a Menu ni a main.